

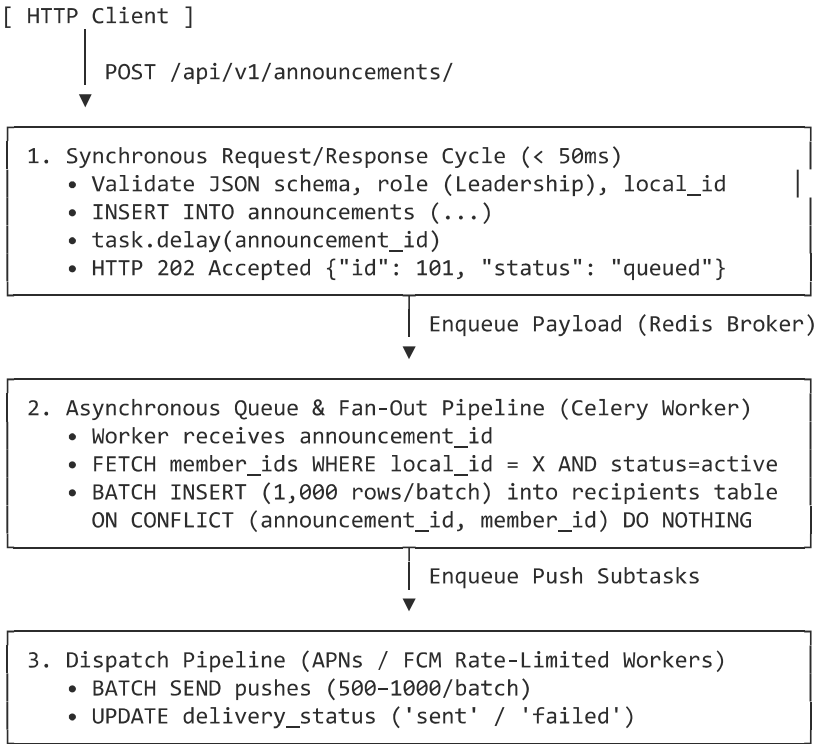
Scaling Announcement Dispatches with Celery

<https://gemini.google.com/app/9fd72728fc4cb0cf>

User prompt: Leadership presses Send for an announcement going to 22,400 people. I want the request handler to only validate the request, create the announcement row, and queue a background job, then return in well under a second. The actual work of creating recipient rows and sending pushes happens later, in a worker, in batches, for example 500 to 1000 rows per batch, not one row at a time. Decision: Celery with Redis as the queue, because it is the standard, well documented pairing with Django, and I do not want to spend exercise time comparing alternatives. Walk me through the full path in order. What happens inside the request and response cycle, and what happens outside it. Tell me if Celery and Redis are overkill here or a reasonable default.

Response: Execution Path: Inside vs. Outside the Request/Response Cycle

When leadership submits an announcement for 22,400 members, the system splits execution into an immediate synchronous cycle and an asynchronous background worker pipeline.



1. Inside the Request/Response Cycle (Synchronous)

Target execution time: ~10–50 ms

- 1. Authentication & Authorization:** Verify that the authenticated user belongs to the local (local_id) and holds a leadership role (role = 'leadership').
- 2. Payload Validation:** Validate the title, body text, and requirements (e.g., needs_ack) against input parameters.
- 3. Database Insertion:** Execute an INSERT query into the announcements table for the specific local_id.
- 4. Enqueue Asynchronous Job:** Call dispatch_announcement.delay(announcement_id). Celery serializes this message payload (containing only the primary key announcement_id) and writes it to a Redis queue.
- 5. Return Immediate HTTP Response:** Return an HTTP 202 Accepted response with the created announcement object metadata ({"id": announcement_id, "status": "queued"}).

2. Outside the Request/Response Cycle (Asynchronous)

Execution time: ~2–10 seconds depending on push provider rate limits

- 1. Task Retrieval:** A Celery worker process fetches the dispatch_announcement task from Redis.
- 2. Batch Recipient Row Generation:**

- Fetch active member IDs for the target `local_id`.
- Query the database in chunks of 1,000 members.

- Bulk-insert records into `announcement_recipients` using batches:

```
INSERT INTO announcement_recipients (announcement_id, member_id, delivery_status)
VALUES (%s, %s, 'pending')
ON CONFLICT (announcement_id, member_id) DO NOTHING;
```

- *Idempotency Protection*: If the process crashes and restarts mid-send, the `UNIQUE (announcement_id, member_id)` constraint prevents duplicate recipient records.

3. Staged Dispatch & Delivery Updates:

- Chunk recipient records into batches of 500–1,000 for push providers (APNs / Google FCM).
- Dispatch notification payloads to third-party push services.
- Update delivery statuses in bulk based on provider responses:

```
UPDATE announcement_recipients
SET delivery_status = 'sent', sent_at = NOW()
WHERE id IN (...);
```

Evaluation: Are Celery and Redis Overkill or a Reasonable Default?

Verdict: A Reasonable, Industry-Standard Default

While simpler alternatives exist, Celery with Redis is a well-balanced choice for this architecture for the following reasons:

- **Separation of Concerns**: Offloading fan-out operations (generating up to 22,400 database records and making thousands of outbound HTTP API calls) guarantees sub-second HTTP response times for leadership.
- **Rate-Limit Management**: Third-party services like Apple APNs or Google FCM enforce connection pooling and throughput limits. Celery natively handles concurrency control and task throttling across worker pools.
- **Failure Recovery & Retry Guarantees**: Network calls fail silently or timeout ~5–10% of the time. Celery provides task retries with exponential backoff and dead-letter queues out of the box.

Lightweight Alternatives Considered

- **Django Q2 / Huey / Dramatiq**: Lighter background task queues that run on Redis or PostgreSQL directly. They offer simpler APIs and smaller footprints than Celery, but don't significantly reduce architectural complexity if Redis is already deployed.
- **PostgreSQL / Database-Backed Queues (e.g., Celery + Postgres Broker or django-tasks)**: Eliminates Redis entirely by using the database as the message queue. While sufficient for small tasks, polling PostgreSQL for thousands of rapid fan-out notifications can introduce write-amplification and lock contention on database tables.

Using Celery + Redis avoids reinventing retry semantics, batch management, or background job scheduling while remaining aligned with standard Django architecture patterns.

User prompt: Four people have the status screen open, watching sent, read, and acknowledged counts update live during a 22,400 person send. If every client polls a live count query against the recipient table once a second, that will overload Postgres. Decision: cache the aggregate counts in Redis with a short expiry, around 3 to 5 seconds, recomputed by the worker as it processes each batch, rather than computing the count fresh from Postgres on every page load, and rather than building websockets. A few seconds of staleness on a status screen is an acceptable tradeoff against the extra build time real time updates would cost. Tell me if that tradeoff makes sense for this exercise, or if websockets would actually be less work than I am assuming

Response: Tradeoff Evaluation: Short-Lived Redis Cache Polling vs. WebSockets

Using a short-lived Redis cache (3–5 seconds) paired with client-side polling is a **reasonable default and the correct tradeoff** for this exercise.

Why Polling with a Short Redis TTL Wins

- 1. **Low Complexity & Fast Build Time:** In Django, setting up a polling endpoint backed by Redis cache (`cache.get_or_set()`) requires writing a standard REST view in ~10 lines of code. WebSockets require asynchronous server orchestration (ASGI / Django Channels), setting up an event bus layer (Redis Channel Layer), running custom server processes (like daphne or uvicorn), and managing client connection lifecycles.
- 2. **Predictable Query Load:** Without caching, 4 clients polling every second generate 4 heavy aggregate queries per second (`SELECT rsvp, count(*) FROM announcement_recipients WHERE announcement_id = %s GROUP BY rsvp`). With a 3-second Redis cache, PostgreSQL receives at most **1 aggregate query every 3 seconds globally**, regardless of how many organizers keep the status screen open.
- 3. **Alignment with Operational Requirements:** Organizers viewing the status screen only need a high-level sense of progress as batches fan out. A 3-to-5-second delay in delivery status updates has zero operational impact on leadership monitoring attendance or callouts.

Would WebSockets Actually Be Less Work?

No. WebSockets would require significantly more infrastructure and boilerplate code.

Dimension	Short Redis Cache + Polling	WebSockets (Django Channels)
Server Infrastructure	Standard WSGI HTTP server (Gunicorn)	ASGI server (Daphne/Uvicorn) alongside WSGI
State & Connections	Stateless HTTP (GET /api/announcements/101/stats/)	Stateful persistent TCP connections & heartbeat management
Worker Integration	Worker updates Redis keys after processing each 500–1,000 row batch	Worker must push events to a Redis channel, broadcast to WebSocket groups
Failure Modes	Standard HTTP retries if a request drops	Connection drops, auto-reconnect logic, missing backpressure controls

Recommended Implementation Strategy

To implement the cached polling approach efficiently:

- 1. **Worker Cache Invalidation/Update:** As the background worker finishes processing a batch of 500–1,000 recipients, it runs the aggregate query and updates Redis with a short time-to-live (TTL):

```
# Inside Celery worker batch loop
stats = {
    "sent": Recipient.objects.filter(announcement_id=id, delivery_status='sent').count(),
    "read": Recipient.objects.filter(announcement_id=id, read_at__isnull=False).count(),
    "acknowledged": Recipient.objects.filter(announcement_id=id, acknowledged_at__isnull=False).count(),
}
cache.set(f"announcement_stats:{id}", stats, timeout=5)
```
- 2. **API Endpoint:** The status API view attempts to read directly from Redis. If the key is missing or expired, it falls back to the database, computes the count, sets the key, and returns the response.
- 3. **Frontend Polling:** The React/Vue dashboard executes a simple `setInterval` fetching the endpoint every 3 to 5 seconds until the job state is marked as complete.